



FINAL PROJECT DEVOPS

Project Tim 1

Nama Mahasiswa

Syifani Adillah Salsabila
Khaelano Abroor Maulana
Muhammad Gathan Raka

Pendamping:

Mba Alfina

Mas Nabih



Poros FILKOM UB
Universitas Brawijaya
2024

(TEAM) FINAL PROJECT RECRUITMENT DEVOPS POROS 2025

Anda harus membangun pipeline CI/CD menggunakan GitHub Actions untuk mendeploy aplikasi web sederhana dalam Docker Swarm dengan minimal 1 manager dan 2 worker nodes.

Link Github Organization: <https://github.com/Final-Project-DevOps>

Link repo FE: <https://github.com/Final-Project-DevOps/Front-End.git>

Link repo BE: <https://github.com/Final-Project-DevOps/Back-end.git>

Link url App: <https://knowhubdevops.netlify.app/> atau <https://fir-react-devops.web.app/>

Link Gdrive: [FINAL_PROJECT_DEVOPS_KELOMPOK1](#)

Spesifikasi

1. Dockerization

- Buat Dockerfile untuk aplikasi web dengan fungsi (**Minimal**) CRUD (bisa berbasis Node.js, Python Flask, atau Golang).
- Pastikan image yang dihasilkan ringan dan efisien.

Note: **Bisa menggunakan multi-stage build untuk membuat image lebih ringan.**

- Push image ke Docker Hub atau GitHub Container Registry.
- Aplikasi harus bisa dijalankan secara lokal dengan docker-compose sebelum di deploy ke Docker Swarm.

Dockerfile Front-end:

```
# Build stage
FROM node:18-alpine3.18 AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Production stage
FROM nginx:latest

# Copy build result ke nginx
COPY --from=builder /app/build /usr/share/nginx/html

# Expose port 80
```

EXPOSE 80

Jalankan nginx

CMD ["nginx", "-g", "daemon off;"]

Dockerfile Back-end:

FROM node:slim AS build

WORKDIR /temp/

COPY package.json package-lock.json ./

RUN npm install

COPY ./ ./

RUN npm run build

FROM node:slim AS deploy

RUN apt-get update -y

RUN apt-get install -y openssl

FROM deploy

WORKDIR /app

COPY --from=build /temp/dist/ ./dist/

COPY --from=build /temp/entrypoint.sh ./

COPY --from=build /temp/prisma/ ./

COPY --from=build /temp/package.json ./

ENV NODE_ENV=production

RUN npm install

RUN chmod +x ./entrypoint.sh

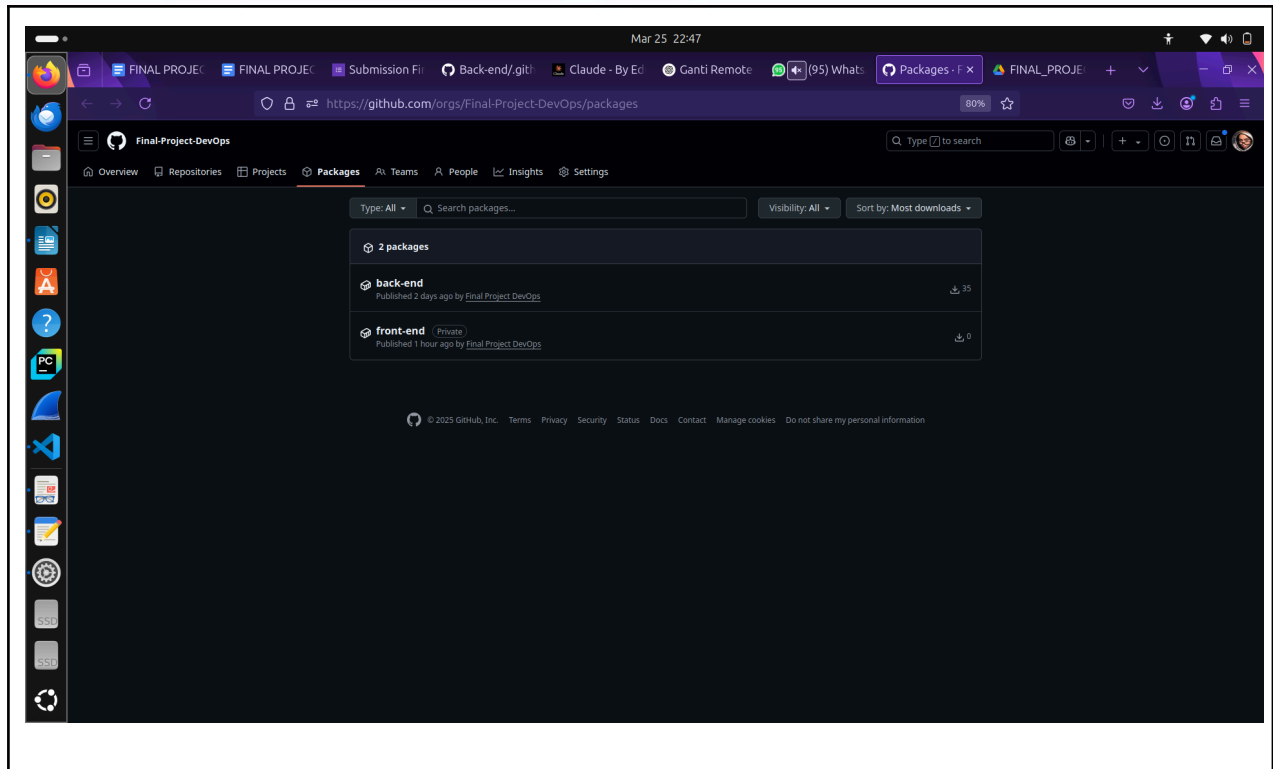
EXPOSE 8080

ENTRYPOINT ["bash", "./entrypoint.sh"]

2. CI/CD dengan GitHub Actions

- Buat workflow yang melakukan build & push Docker image secara otomatis ketika ada perubahan pada branch utama.

Url Package: <https://github.com/orgs/Final-Project-DevOps/packages>



CI/CD Front-end:

name: "Build and Publish Frontend Image to GHCR"

on:

workflow_dispatch:

push:

branches:

- master

env:

REGISTRY: ghcr.io

IMAGE_NAME: ghcr.io/final-project-devops/front-end

jobs:

build:

name: "Build Frontend Base Image"

runs-on: "ubuntu-24.04"

steps:

- name: "Checkout Repository with Custom Depth"

uses: actions/checkout@v4

with:

fetch-depth: 2

- name: "Set Up Docker Buildx"

uses: docker/setup-buildx-action@v3

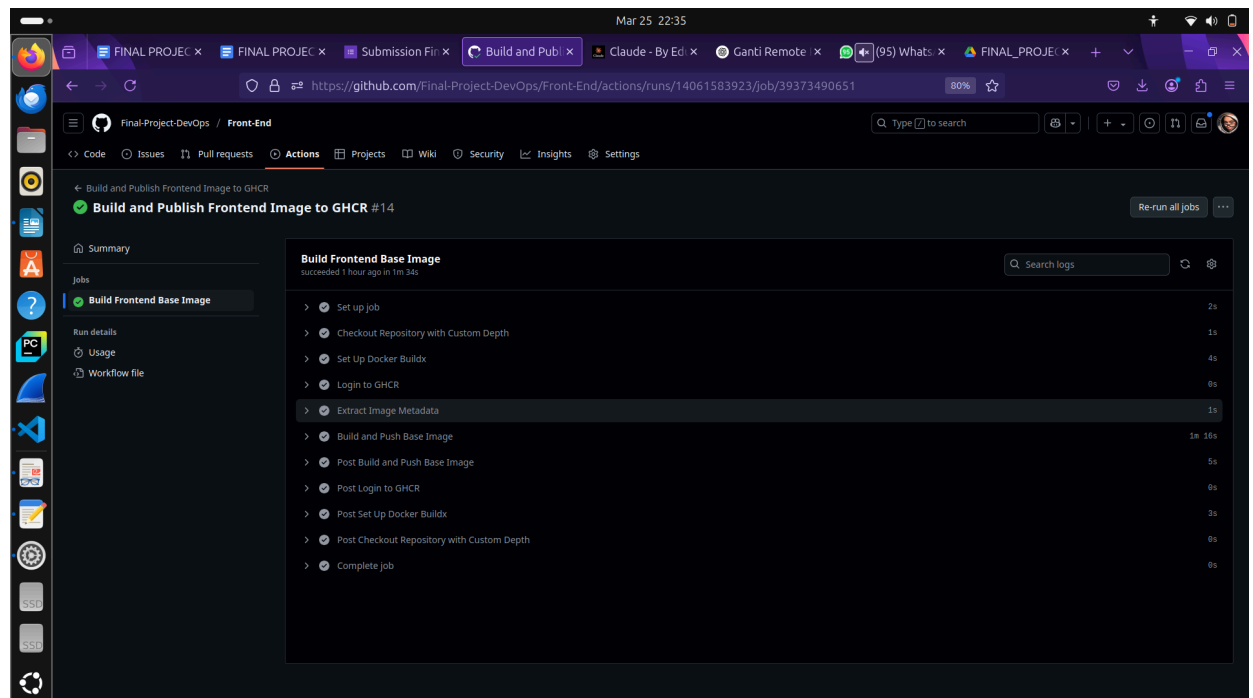
```

- name: "Login to GHCR"
  uses: docker/login-action@v3
  with:
    registry: ${ env.REGISTRY }
    username: ${ github.actor }
    password: ${ secrets.GHCR_TOKEN }

- name: "Extract Image Metadata"
  id: metadata
  uses: docker/metadata-action@v5
  with:
    images: ${ env.REGISTRY }/${ env.IMAGE_NAME }

- name: "Build and Push Base Image"
  uses: docker/build-push-action@v6
  with:
    context: .
    file: Dockerfile
    push: true
    tags: ${ steps.metadata.outputs.tags }
    labels: ${ steps.metadata.outputs.labels }
    cache-from: type=gha
    cache-to: type=gha,mode=max

```



CI/CD Back-end:

```

name: "Build and Publish Image to GHCR"

```

```
on:
  push:
    branches:
      - master

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${github.repository}

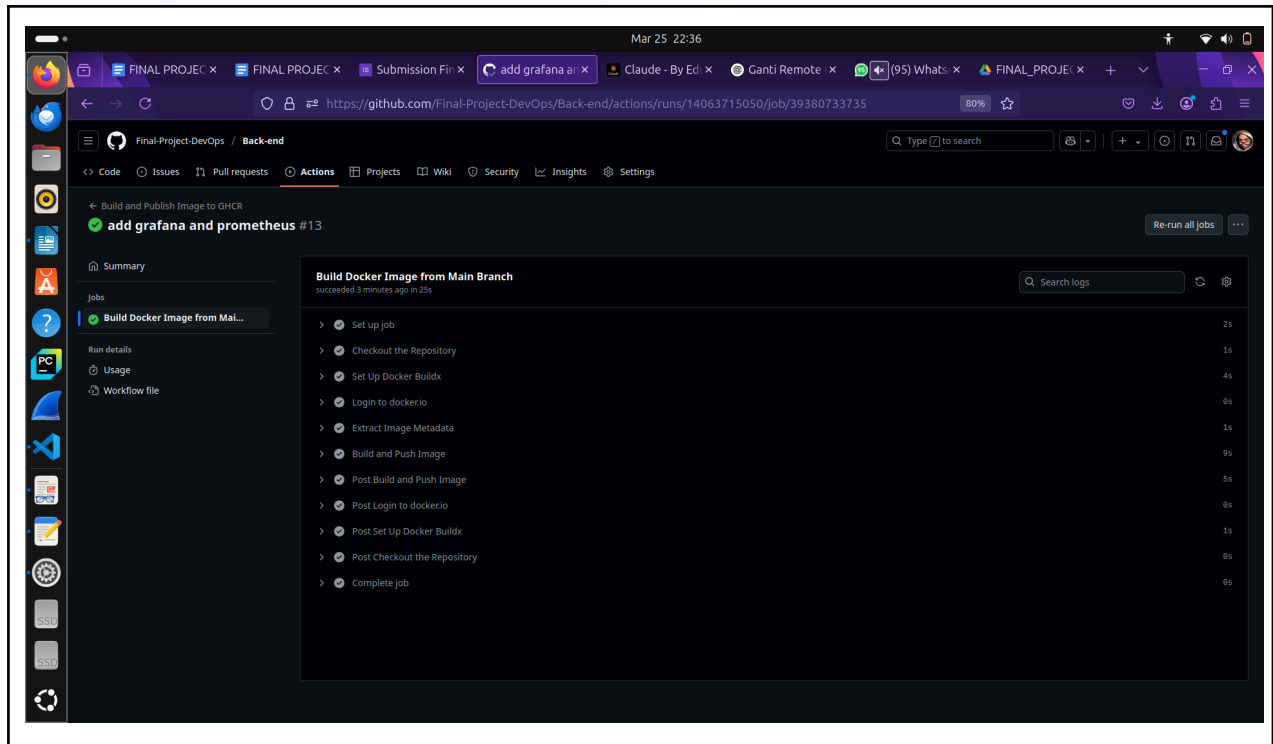
jobs:
  build:
    name: "Build Docker Image from Main Branch"
    runs-on: "ubuntu-24.04"
    steps:
      - name: "Checkout the Repository"
        uses: actions/checkout@v4

      - name: "Set Up Docker Buildx"
        uses: docker/setup-buildx-action@v3

      - name: "Login to docker.io"
        uses: docker/login-action@v3
        with:
          registry: ${env.REGISTRY}
          username: ${github.actor}
          password: ${secrets.GHCR_TOKEN}

      - name: "Extract Image Metadata"
        id: metadata
        uses: docker/metadata-action@v5
        with:
          images: ${env.REGISTRY}/${env.IMAGE_NAME}

      - name: "Build and Push Image"
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${steps.metadata.outputs.tags}
          labels: ${steps.metadata.outputs.labels}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```



3. Docker Swarm Deployment (**Minimal** : 1 Manager Node, 2 Worker Nodes)

- Deploy aplikasi sebagai service menggunakan Docker Stack.
- Manager Node tidak boleh ikut menjalankan task.
- Implementasikan Docker Volume.
- Konfigurasi network supaya stack yang dijalankan tidak menggunakan network overlay default.

```
ubuntu@ip-172-31-40-19:~$ docker swarm init --advertise-addr 172.31.40.19
Swarm initialized: current node (2wrhesymxy6da56add6xz5cgo) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-08awplyjiwffjdnzebek8dikuvpzjelvu102qxve7nzriiqww0-7xr88c9cqgefna0qihsh5biuk 172.31.40.19:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

ubuntu@ip-172-31-29-92:~$ docker swarm join --token SWMTKN-1-08awplyjiwffjdnzebek8dikuvpzjelvu102qxve7nzriiqww0-7xr88c9cqgefna0qihsh5biuk 172.31.40.19:2377
This node joined a swarm as a worker.

ubuntu@ip-172-31-18-137:~$ docker swarm join --token SWMTKN-1-08awplyjiwffjdnzebek8dikuvpzjelvu102qxve7nzriiqww0-7xr88c9cqgefna0qihsh5biuk 172.31.40.19:2377
This node joined a swarm as a worker.

ubuntu@ip-172-31-40-19:~$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
4k6cx47d1o9km6b6hnsjrmk38	ip-172-31-18-137	Ready	Active		28.0.2
qzf5p7wubrb8b5yd5kn6fy1im	ip-172-31-29-92	Ready	Active		28.0.2
2wrhesymxy6da56add6xz5cgo *	ip-172-31-40-19	Ready	Active	Leader	28.0.2

```

ubuntu@ip-172-31-40-19:~$ docker info
Client: Docker Engine - Community
Version: 28.0.2
Context: default
Debug Mode: false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version: v0.22.0
    Path: /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version: v2.34.0
    Path: /usr/libexec/docker/cli-plugins/docker-compose

Server:
Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
Images: 0
Server Version: 28.0.2
Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
Swarm: active
  NodeID: 2wrhesymxy6da56add6xz5cgo
  Is Manager: true
  ClusterID: eekg9q5k4k9lgftv48zozlb0z
  Managers: 1
  Nodes: 3
  Data Path Port: 4789
  Orchestration:
    Task History Retention Limit: 5
  Raft:
    Snapshot Interval: 10000
    Number of Old Snapshots to Retain: 0
    Heartbeat Tick: 1
    Election Tick: 10

```

==

4. Manajemen Konfigurasi & Secret

- Simpan environment variables dalam Docker Config dan credentials ke dalam Docker Secret.
- Gunakan cara yang aman dalam pengelolaan secret (tidak hardcoded dalam YAML, etc.,)



```

ubuntu@ip-172-31-40-19:~/Back-end$ echo "2332KerenBang." | docker secret create postgres_password -
v2lb8tjl1gi2ndd6a6gqntf4x
ubuntu@ip-172-31-40-19:~/Back-end$ echo "postgresql://khaelano:2332KerenBang.@db:5432/serverdb?schema=public" | docker secret create db_url -
h43gt187pit03tn1x1kos7tup
ubuntu@ip-172-31-40-19:~/Back-end$ echo "khaelano" | docker      create postgres_user -
r8otdq0xg9esakcl6k347r7j0

```

5. Monitoring dan Logging

- Integrasikan dengan Prometheus + Grafana atau ELK Stack untuk monitoring/logging dasar.

Note: pastikan minimal bisa menjalankan Prometheus + Grafana (atau sebagainya sesuai preferensi) secara lokal dalam cluster.

```

ubuntu@ip-172-31-40-19:~/Back-end$ docker stack services back-end

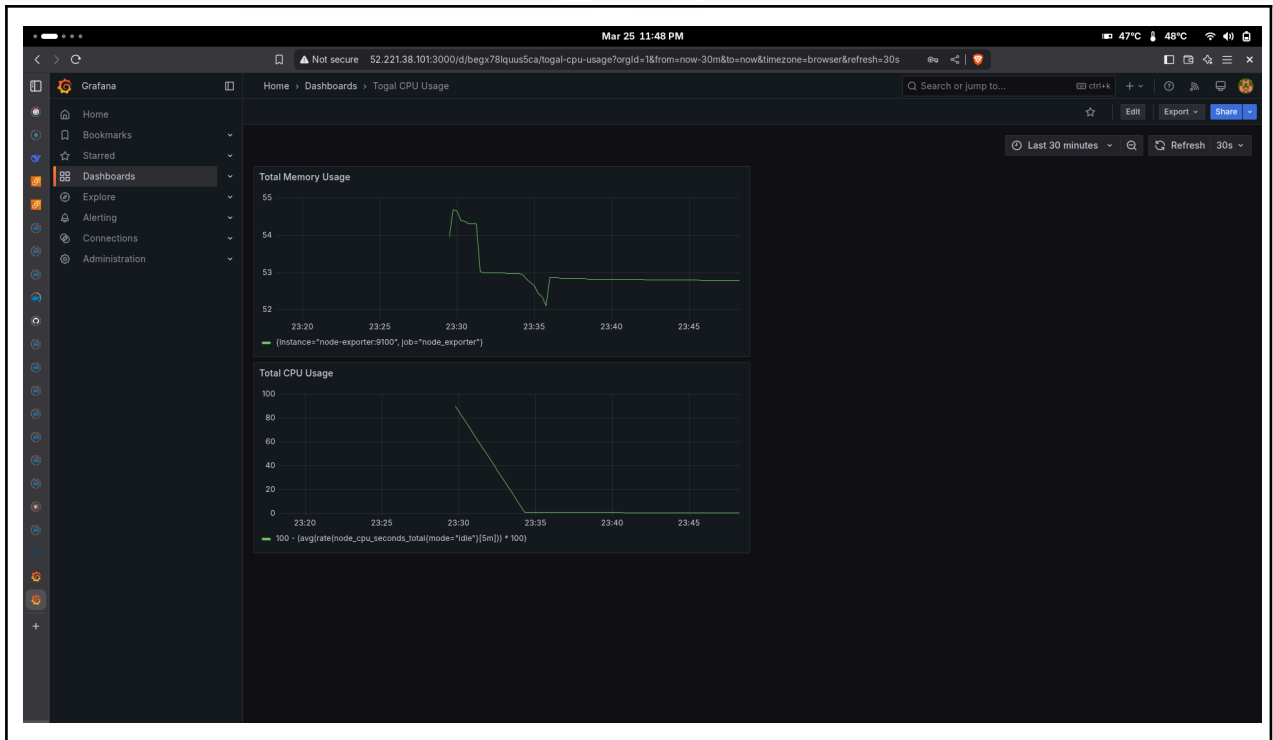
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
il2zvagd8tmzz	back-end_db	replicated	1/1	postgres:latest	*:5432->5432/tcp
24znas8skivr	back-end_grafana	replicated	1/1	grafana/grafana:latest	*:3000->3000/tcp
zh0ehihu8ax6	back-end_node-exporter	global	2/2	prom/node-exporter:latest	
iz5v0sft1u7j	back-end_prometheus	replicated	1/1	prom/prometheus:latest	*:9090->9090/tcp
bw1arpebjfv2	back-end_server	replicated	3/3	ghcr.io/final-project-devops/back-end:master	*:8080->80/tcp

```

ubuntu@ip-172-31-40-19:~/Back-end$

```



prometheus.yml:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "node_exporter"
    static_configs:
      - targets: ["node-exporter:9100"]
```

docker-compose.yml:

```
services:
  server:
    image: ghcr.io/final-project-devops/back-end:master
    entrypoint: ["/bin/sh", "-c", "export DATABASE_URL=\\$(cat /run/secrets/db_url) && ./entrypoint.sh"]
    ports:
      - 8080:80
    networks:
      - internal-net
    secrets:
      - db_url
    deploy:
      replicas: 3
    depends_on:
      - db
```

db:

```
image: postgres:latest
volumes:
- pgdata:/var/lib/postgresql/data
ports:
- 5432:5432
environment:
POSTGRES_USER_FILE: /run/secrets/postgres_user
POSTGRES_PASSWORD_FILE: /run/secrets/postgres_password
networks:
- internal-net
secrets:
- postgres_user
- postgres_password
```

prometheus:

```
image: prom/prometheus:latest
configs:
- source: prometheus_config
target: /etc/prometheus/prometheus.yml
ports:
- 9090:9090
networks:
- internal-net
```

grafana:

```
image: grafana/grafana:latest
ports:
- 3000:3000
volumes:
- grafana_data:/var/lib/grafana
networks:
- internal-net
```

node-exporter:

```
image: prom/node-exporter:latest
deploy:
mode: global
networks:
- internal-net
```

configs:

```
prometheus_config:
file: ./prometheus.yml
```

secrets:

```
postgres_user:
external: true
```

<pre> postgres_password: external: true db_url: external: true networks: internal-net: driver: overlay volumes: pgdata: grafana_data: </pre>
--

Penilaian

Kategori	Bobot (%)	Kriteria Penilaian
Dockerization	20%	Struktur Dockerfile, efisiensi ukuran image, best practices
CI/CD Pipeline	25%	Workflow GitHub Actions, build automation, deployment
Kubernetes Deployment	30%	Struktur file compose dan konfigurasi cluster
Konfigurasi & Secret	15%	Penggunaan Docker Config dan Docker Secret yang baik dan benar
Monitoring	10%	Implementasi monitoring dan logging

Pengumpulan

Buatkan sebuah drive yang berisi :

- Laporan dokumentasi (.pdf) berisi dokumentasi dan laporan, sertakan link repo GitHub proyek didalamnya, sertakan juga URL app **jika berhasil berjalan pada cloud**
- Environment file (.env)
- File compose

Dateline pengumpulan : (25 Maret 2025, 23.59)

Form Pengumpulan: (<https://forms.gle/DrogrGkZSEvDiuL48>)

Catatan

- Pastikan semua konfigurasi bisa direplikasi dengan baik untuk penilaian (**Wajib**).
- Implementasikan Docker Scout untuk scanning vulnerability pada workflow (**nilai +**).
- Kompleksitas Fungsional dan Fitur aplikasi akan memberikan (**nilai +**).
- Menggunakan Terraform atau Ansible untuk setup infrastruktur (**nilai +**).
- Bonus poin jika menggunakan Helm Chart untuk deployment (**nilai +**).